

研究関連説明

2024.12.06

cm24026 田上 雄悟

目次

1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

2. 音声処理の基礎

1. 音声処理とは
2. 音声信号の特徴
3. 特徴量抽出

3. 実際にやってみよう

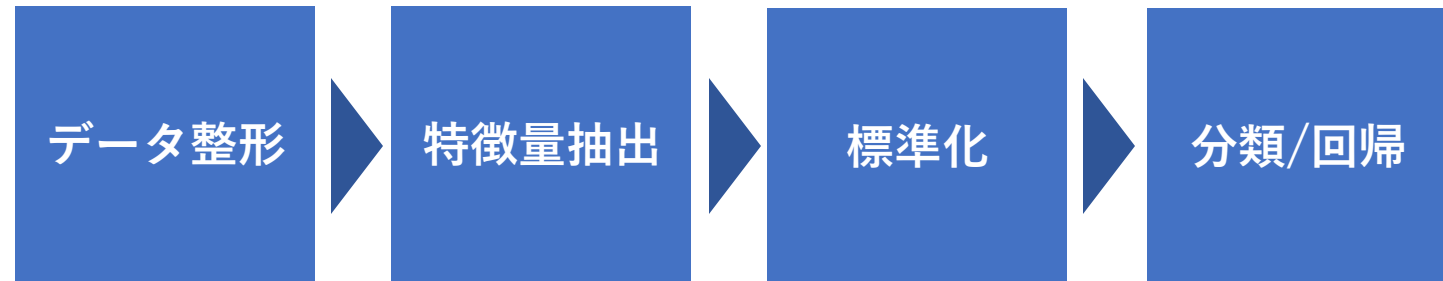
1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

大量のデータからパターンやルールを学び、それを基に新しいデータに対して予測を行う

機械学習は数学と統計に基づいている

1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers



1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

機械学習(ML)では主に「教師あり学習」と「教師なし学習」がある

教師あり学習 (supervised learning) :

入出力のデータを使用してモデルを構築

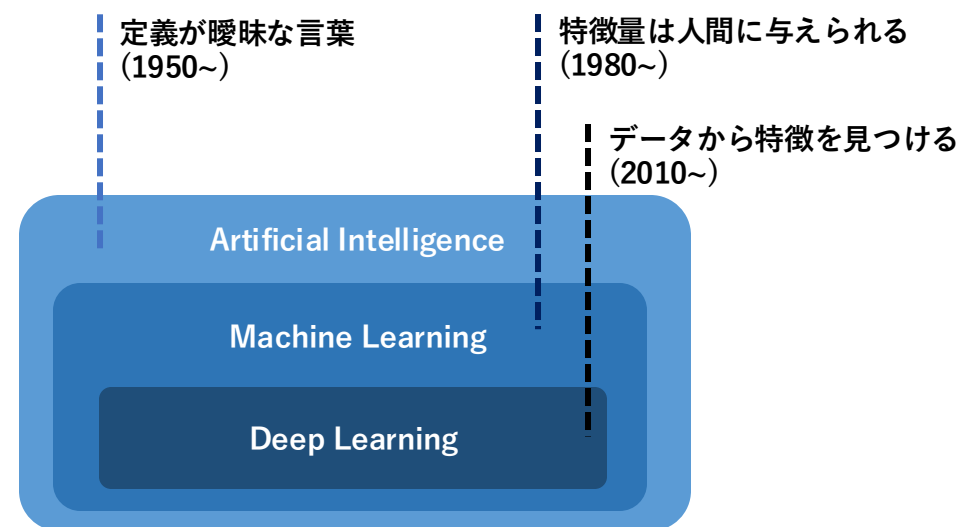
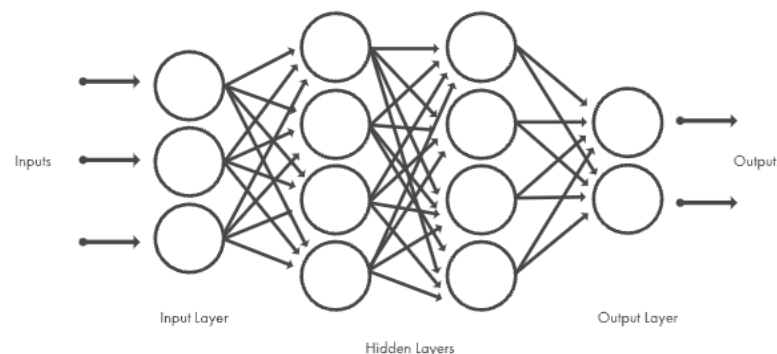
– 分類 (SVM, 最近傍法) / 回帰 (線形回帰, アンサンブル木, NN)

教師なし学習 (unsupervised learning) :

入力 of データに基づいてグループ化する

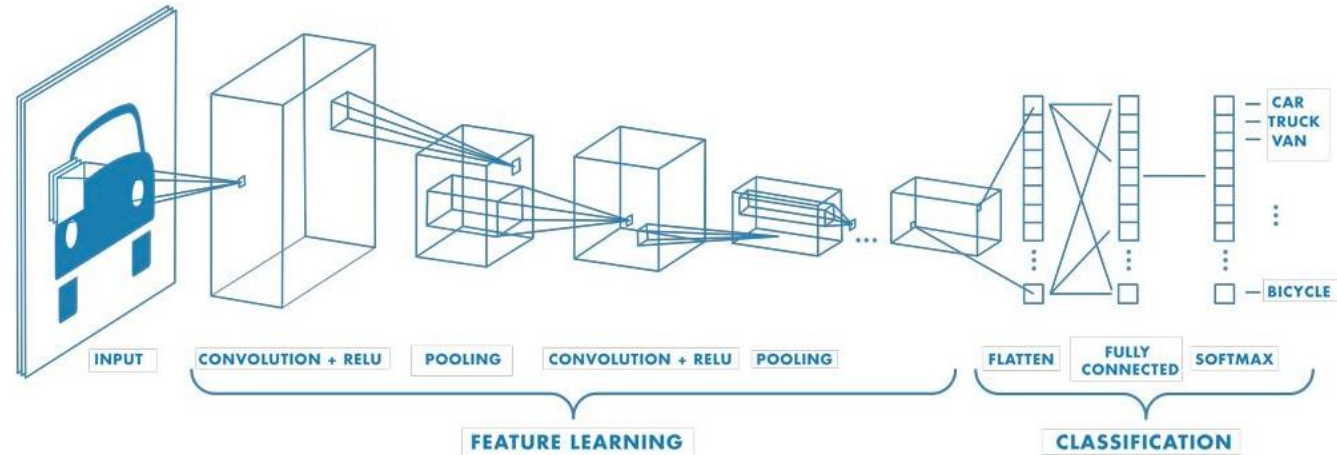
– クラスタリング (k-means, HMM, 混合ガウスモデルなど)

深層学習(DL)では複数の隠れ層を持つニューラルネットワークで構成される

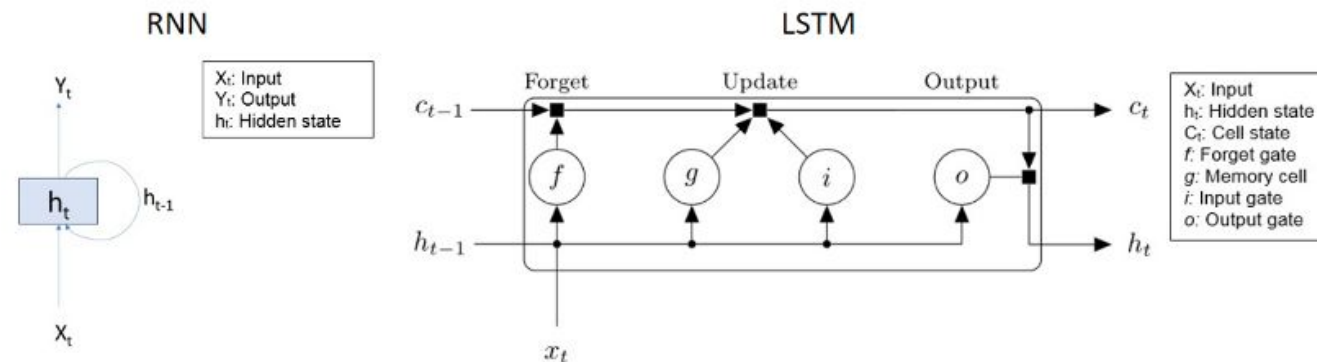


深層学習(DL)にはCNN, RNN, Transformerモデルの3種類がある

CNN (畳み込みニューラルネットワーク)



RNN (リカレントニューラルネットワーク)



Transformerについては後で説明

1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

すべての機械学習モデルは数値以外のデータを扱うことができない

数値、テキスト、画像、音声、その他のデータはすべてモデルへの入力までに何らかの処理を行なって数字に変換される

どのように数字に変換するかが性能に大きく関係する

機械学習は万能ではない

不適切なデータが学習に用いられると適切な予測をすることができない

データを分析して不適切なデータを排除したり、相関係数を調べたりして説明変数を選んだり、**標準化**(Scaling)・**正規化**(Normalization)、**正則化**(Regularization)をする

タスク(目的)に応じて使用するアルゴリズムやモデルの構造を変える必要がある

データセット

学習用データセット(Training set)、**検証用**データセット(Validation set)、**テスト用**データセット(Test set)がある
例えば学習用データセットに

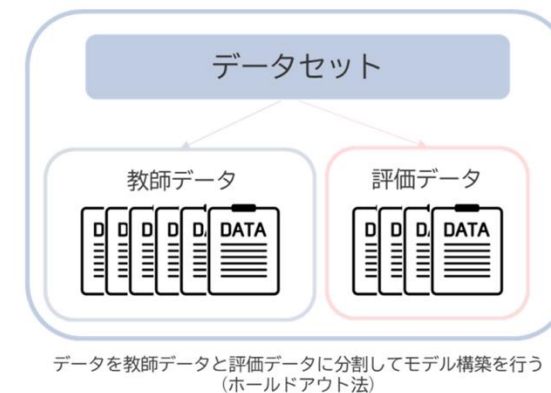
1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

モデルの学習・評価の手法

ホールドアウト法

データセットを訓練データセットと検証データセットに分割してモデルの精度を評価する手法

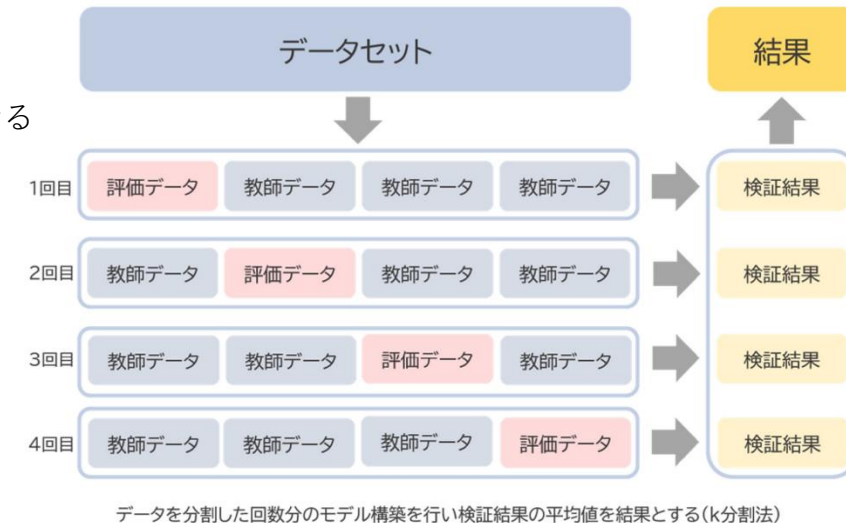


交差検証 (Cross Validation)

データセットを繰り返し分割して複数のモデルを訓練して評価する

- K-分割交差検証

データセットをK個に分割し、学習データセットとして使うものと検証データセットとして使うものを変化させる



1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

モデルの学習(応用)

転移学習

ある問題で学習した特徴量を取り入れ、それを新しい類似した問題に利用する方法。

- ファインチューニング

既存の学習済みモデルの一部または全部の重みを非常に低い学習率で追加学習すること。

ラベルの数値化(エンコーディング)

カテゴリ変数(順位や性別など)を数値に変換すること

one-hot encoding

カテゴリの数だけ変数を作り、各データについて対応するカテゴリの変数を1に、それ以外を0にする

label encoding

カテゴリに対応する値を用意し、各データについてカテゴリを対応する値に置き換える

frequency encoding

カテゴリを、そのカテゴリが出現する頻度に置き換える

モデルの学習ではモデルが説明変数から目的変数を適切に予測できるようにする

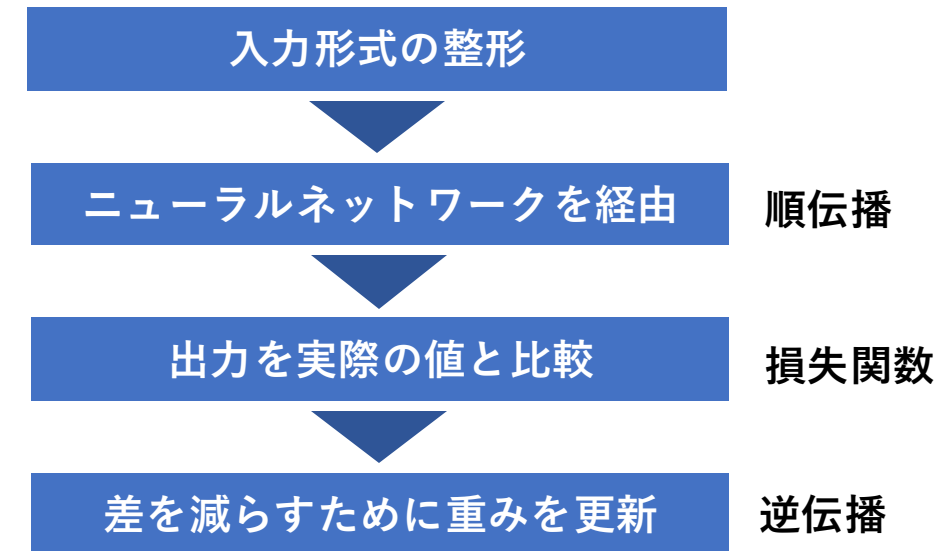
予測がどの程度適切かを評価するためにモデルの出力と実際の値を比較する

1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

- 回帰モデルではMAE(平均絶対誤差), MSE(平均二乗誤差), RMSE(二乗平均平方根誤差), 決定係数(R^2)などをメトリックとして使用することが多い
- 分類モデルでは混合行列を基に、正解率, 適合率, リコール, F1スコア等が使用される

深層学習での学習方法について



学習回数(epoch)

学習データ全体を学習する回数。モデルの大きさや損失、精度を評価して調整する。

バッチサイズ(batchsize)

1回のパラメータ更新でモデルに入力するデータの数。小さいとノイズに敏感になり、大きいと1回の更新に時間がかかる。

損失関数(loss function)

どれくらい正解から離れているかを表す(計算結果は小さいほどいい)

- **Cross Entropy**: 2クラスの分類モデルで用いられることが多い
- **KL divergence**: 生成系モデルで使用される
- **Binary Cross Entropy**: 2クラス分類で用いられることが多い

学習率(learning rate)

パラメータをどのくらい更新するか。大きすぎると不安定になり、小さすぎると学習が進まない

最適化手法(optimizer)

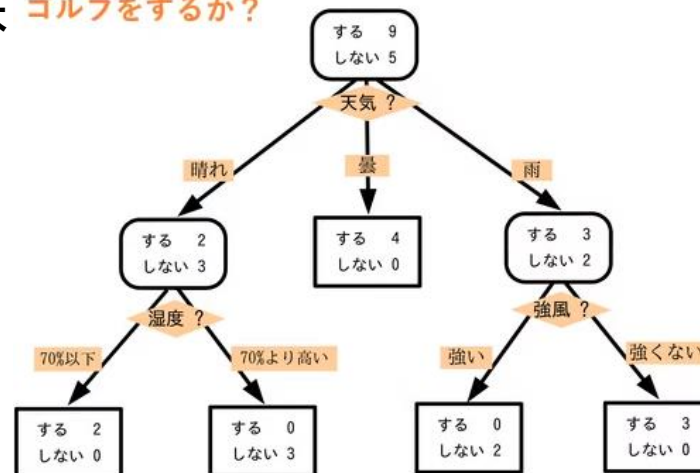
モデルのパラメータ更新をどのように行うか。モデルの損失関数を最小にするように働く(*後で説明)

機械学習のアルゴリズム

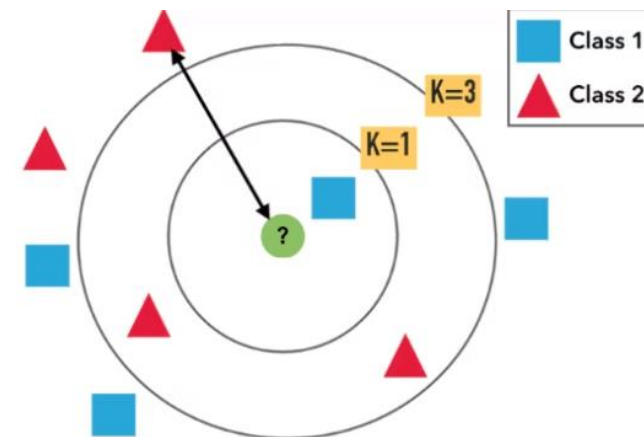
1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

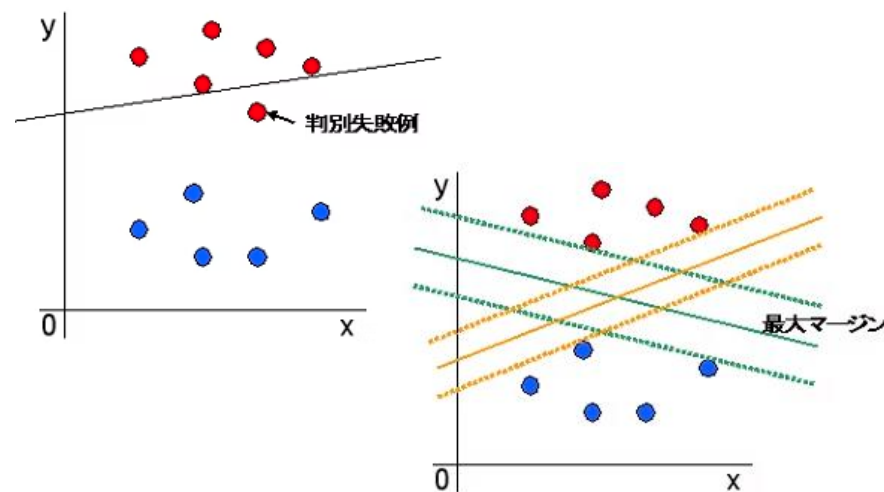
決定木 ゴルフをするか？



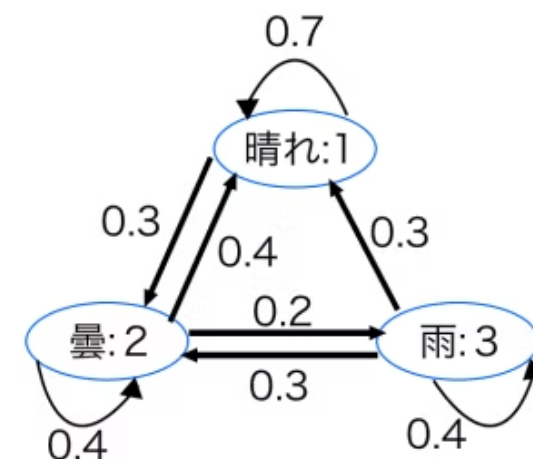
K-means法



サポートベクターマシン



マルコフ連鎖



ニューラルネットワークの最適化アルゴリズム

1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

最急降下法 : 損失関数をパラメータで微分して最小値に近づける(全データを入力する)

確率的勾配降下法(SGD) : ランダムに1つデータを選んで勾配を更新する

モーメンタム : 勾配の移動平均を使用して更新する(振動しにくくなる)

RMSProp : 勾配の大きさに応じて学習率を調整する(振動時に学習が進みにくくなる)

Adam : モーメンタムとRMSPropの組み合わせ

ニュートン法 : パラメータ更新に2階微分を使用(計算量が大きい)

1. 機械学習の基礎

1. 機械学習とは
2. 主な種類
3. データの重要性
4. モデルの学習と評価
5. 主なアルゴリズム
6. Transformers

大規模言語モデル

2023年度統計関連学会連合大会チュートリアルセッション
言語モデルと自然言語処理のフロンティア

岡崎 直観



東京工業大学
Tokyo Institute of Technology

東京工業大学
情報理工学院

okazaki@c.titech.ac.jp
<https://www.nlp.c.titech.ac.jp/>



OKAZAKILAB

音声処理(Speech Processing)は音声をデジタル信号として分析、変換、操作すること

- 人間の声は連続的に変化するアナログ信号である
- コンピュータで扱うにはアナログ信号を数値化し、**デジタル信号に変換**する必要がある
- アナログ信号の変換には**サンプリング**と**量子化**を行う
- 音声信号には**時間次元**と**周波数次元**がある

2. 音声処理の基礎

1. 音声処理とは
2. 音声信号の特徴
3. 特徴量抽出

音声信号は以下の特徴を持つ

- 時間軸に依存する(順番が重要)
- 周波数成分が重要(高さ、種類は波形からわからない)
- サンプル数が多い(16kHzであれば1秒間に16000のデータを持つ)
- 環境ノイズの影響を受けやすい(特定の周波数や時間にノイズがあることがある)
- 音素・声質・ピッチ・話し方など要素が多いため複雑
- 人間の聴覚に基づいたスケールは振幅値と異なる(扱いに慣れる必要がある)

2. 音声処理の基礎

1. 音声処理とは

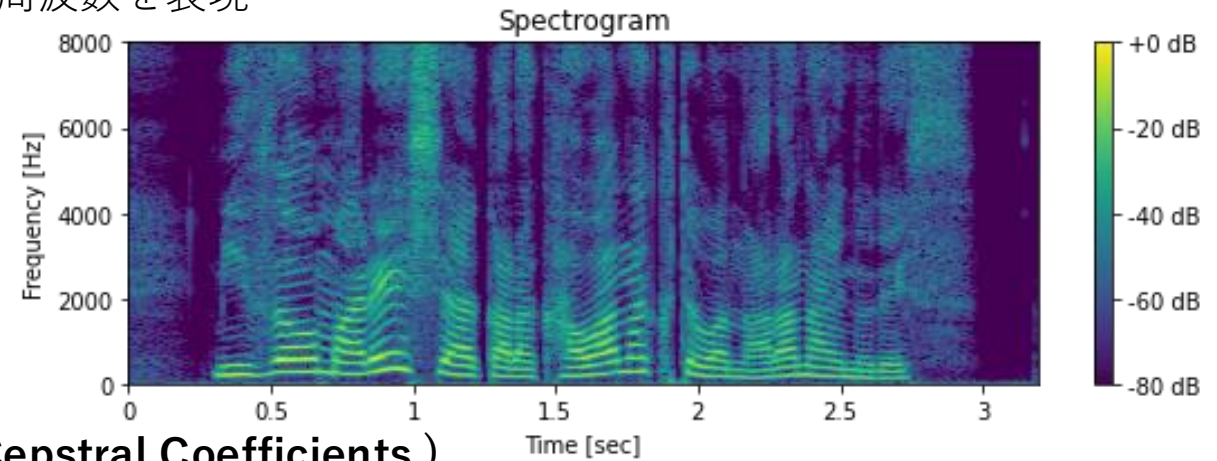
2. 音声信号の特徴

3. 特徴量抽出

音声の特徴量の抽出には主に以下の方法がある

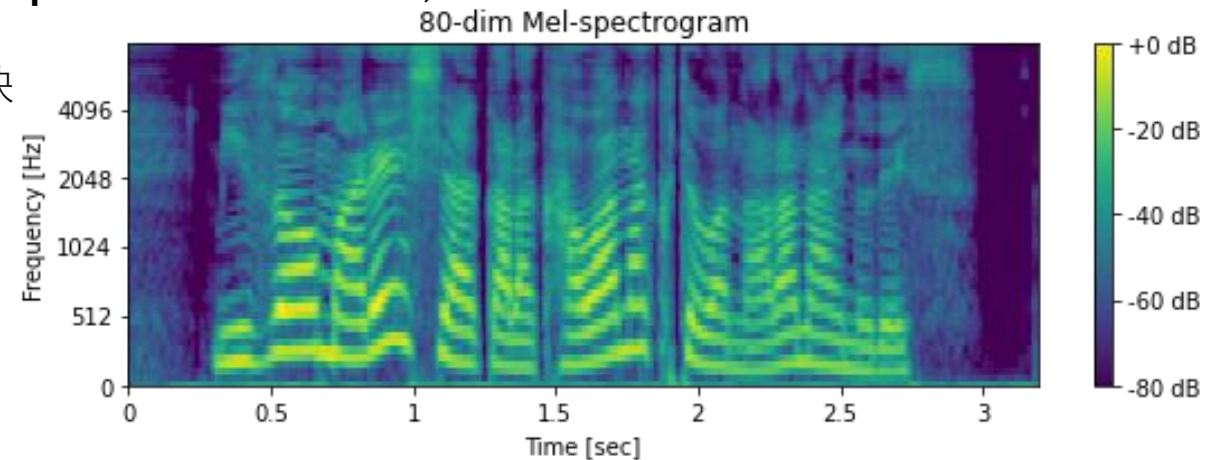
スペクトログラム

- STFTを使用して時間と周波数を表現



MFCC(Mel-Frequency Cepstral Coefficients)

- 人間の聴覚の特性を反映



2. 音声処理の基礎

1. 音声処理とは
2. 音声信号の特徴
3. 特徴量抽出

Pythonの機械学習ライブラリにはいくつかの種類ある

- PyTorch(Meta) : オープンソース、モデルの変更が簡単、GPUサポート、DL向き
- TensorFlow(Google) : オープンソース、さまざまなAPIを用意、シンプルで直感的
- Scikit-learn : 古典的な機械学習アルゴリズム、前処理機能やモデル評価に便利

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
```

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

Iris データセット (有名な花の分類データセット) を使用して
アヤメの種類 (setosa, versicolor, virginica) の **3クラスに分類**

インポート

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
```

データをモデルに入力できる形にする

```
# データ準備
iris = load_iris()
X = iris.data # 特徴量
y = iris.target # ラベル

# データ分割
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# データの標準化
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# データをテンソルに変換
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.long)
```

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

モデルの定義、損失関数、オプティマイザの設定

```
class IrisClassifier(nn.Module):
    def __init__(self):
        super(IrisClassifier, self).__init__()
        self.fc1 = nn.Linear(4, 16) # 入力次元: 4, 隠れ層: 16
        self.fc2 = nn.Linear(16, 3) # 隠れ層: 16, 出力次元: 3 (クラス数)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.fc2(x) # 出力層にSoftmaxは含めない (損失関数で処理)
        return x

# モデル、損失関数、オプティマイザの設定
model = IrisClassifier()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

モデルのトレーニング

```
epochs = 100
for epoch in range(epochs):
    optimizer.zero_grad()
    outputs = model(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')
```

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

モデルの評価

```
with torch.no_grad():  
    y_pred = model(X_test)  
    y_pred_classes = torch.argmax(y_pred, axis=1)  
    accuracy = (y_pred_classes == y_test).sum().item() / y_test.size(0)  
    print(f'Accuracy: {accuracy:.4f}')
```

自分でデータを入力してみる

```
data = [0.3545, -0.5851, 0.5578, 0.0222]  
value = torch.tensor(data, dtype=torch.float32)  
model.eval()  
output = model(value)  
pred_classes = int(torch.argmax(output, axis=0))  
print(output)  
print("クラス:", pred_classes)
```

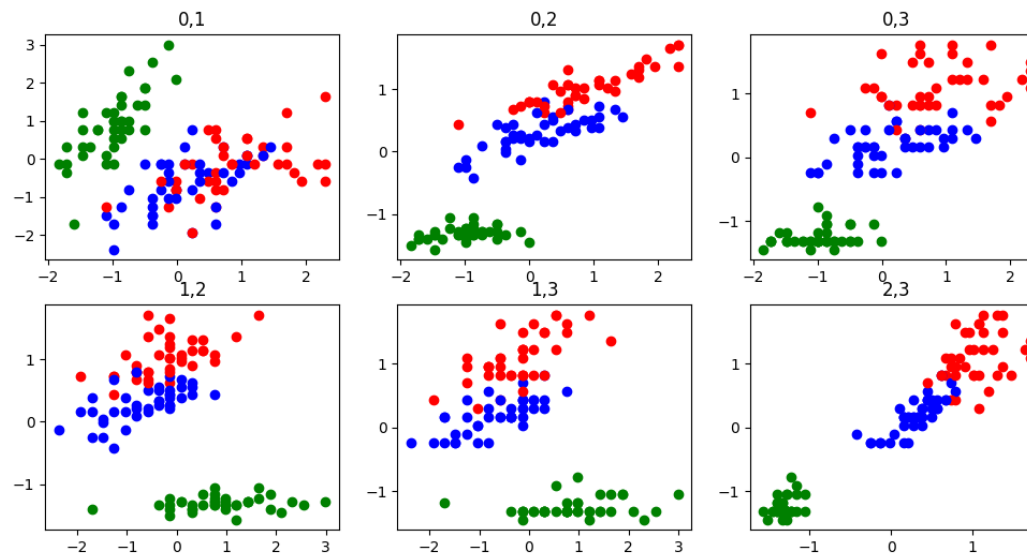
出力例

```
Epoch [10/100], Loss: 0.7431  
Epoch [20/100], Loss: 0.4599  
Epoch [30/100], Loss: 0.3370  
Epoch [40/100], Loss: 0.2670  
Epoch [50/100], Loss: 0.2095  
Epoch [60/100], Loss: 0.1516  
Epoch [70/100], Loss: 0.1116  
Epoch [80/100], Loss: 0.0892  
Epoch [90/100], Loss: 0.0755  
Epoch [100/100], Loss: 0.0674  
Accuracy: 1.0000
```

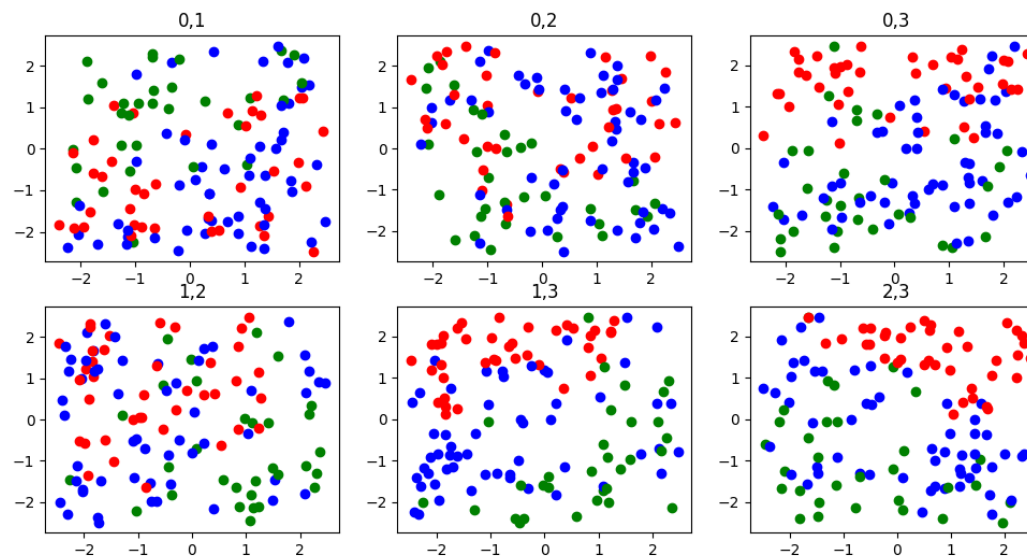
3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

学習データを確認してみる



ランダムなデータを入力してモデルの出力を確認してみる



研究での実験においては再現性が重要なため、乱数値を使用する場合はseed値を固定することが望ましい

境界線がはっきりしない



California housing (住宅価格の予測データセット)を使用して
入力特徴量から**住宅価格（連続値）を予測**

インポート

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

データの準備

```
california_housing = fetch_california_housing()
X = pd.DataFrame(california_housing.data, columns=california_housing.feature_names) # 特徴量
y = california_housing.target

# データ分割
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# データの標準化
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# データをテンソルに変換
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).view(-1, 1) # 出力を列ベクトル化
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).view(-1, 1)
```

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

モデルの定義、損失関数、オプティマイザの設定

```
class BostonRegressor(nn.Module):
    def __init__(self):
        super(BostonRegressor, self).__init__()
        self.fc1 = nn.Linear(8, 128) # 入力次元: 8, 隠れ層: 64
        self.fc2 = nn.Linear(128, 64) # 入力次元: 64, 隠れ層: 64
        self.fc3 = nn.Linear(64, 16) # 入力次元: 64, 隠れ層: 8
        self.fc4 = nn.Linear(16, 1) # 出力次元: 1
        self.bn = nn.BatchNorm1d(16)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        #x = torch.relu(self.fc3(x))
        x = self.bn(self.fc3(x))
        x = self.fc4(x) # 出力層
        return x

# モデル、損失関数、オプティマイザの設定
model = BostonRegressor()
criterion = nn.MSELoss() # 平均二乗誤差
optimizer = optim.Adam(model.parameters(), lr=0.03)
```

モデルのトレーニング

```
epochs = 1000
for epoch in range(epochs):
    optimizer.zero_grad()
    outputs = model(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()

    if epoch==0 or epoch==10 or (epoch + 1) % 100 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')
```

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

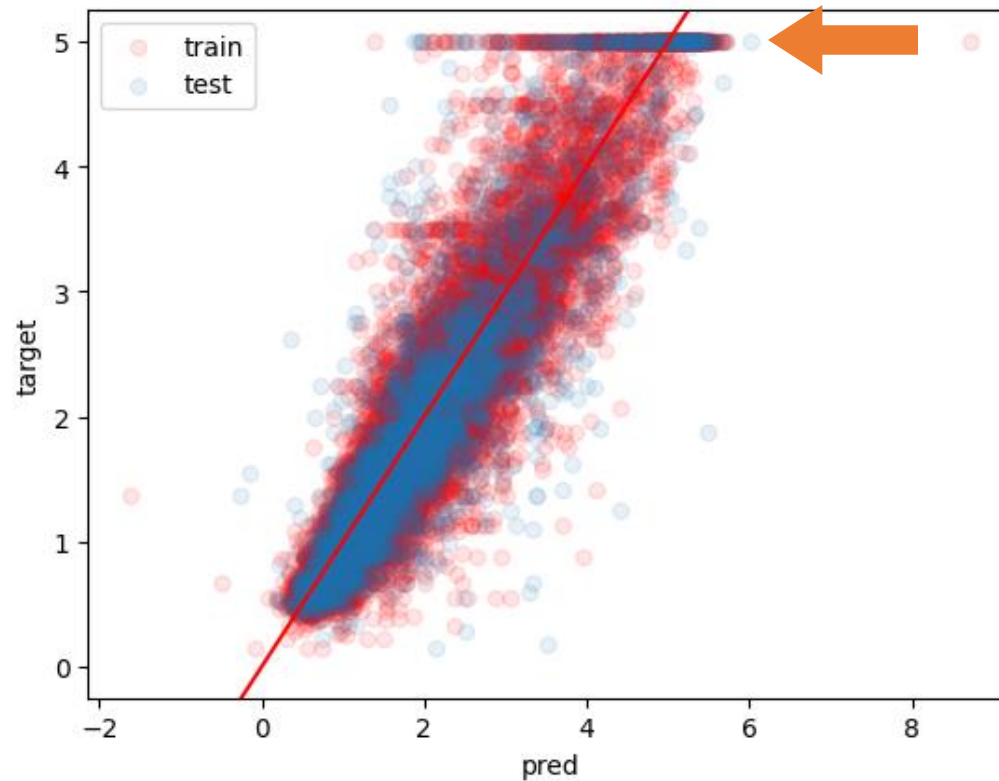
モデルの評価

```
with torch.no_grad():  
    model.eval()  
    y_pred = model(X_test)  
    mse = criterion(y_pred, y_test)  
    print(f'Mean Squared Error(for test): {mse.item():.4f}')
```

出力例

Epoch [1/1000], Loss: 4.9266
Epoch [11/1000], Loss: 1.1819
Epoch [100/1000], Loss: 0.2919
Epoch [200/1000], Loss: 0.2625
Epoch [300/1000], Loss: 0.2356
Epoch [400/1000], Loss: 0.2276
Epoch [500/1000], Loss: 0.2247
Epoch [600/1000], Loss: 0.2099
Epoch [700/1000], Loss: 0.2068
Epoch [800/1000], Loss: 0.1939
Epoch [900/1000], Loss: 0.1900
Epoch [1000/1000], Loss: 0.1891
Mean Squared Error(for test): 0.2833

モデルの出力を確認してみる



target=5のとき予測がおかしい



目標：音声ファイルからpythonで特徴量を抽出してモデルに入力できるようにする

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

インポート

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np
```

スペクトログラムに変換

```
# 音声ファイルのパス
audio_path = "testaudio.mp3"

# 音声ファイルの読み込み
# y: 音声信号（波形データ）, sr: サンプルレート
y, sr = librosa.load(audio_path, sr=22050) # sr=Noneを指定すると元のサンプルレートが維持されます

# 短時間フーリエ変換（STFT）を計算
stft = librosa.stft(y, n_fft=2048, hop_length=512)

# 振幅スペクトル（絶対値を取る）
spectrogram = np.abs(stft)

# 対数振幅スペクトル（dB単位）
log_spectrogram = librosa.amplitude_to_db(spectrogram)
```

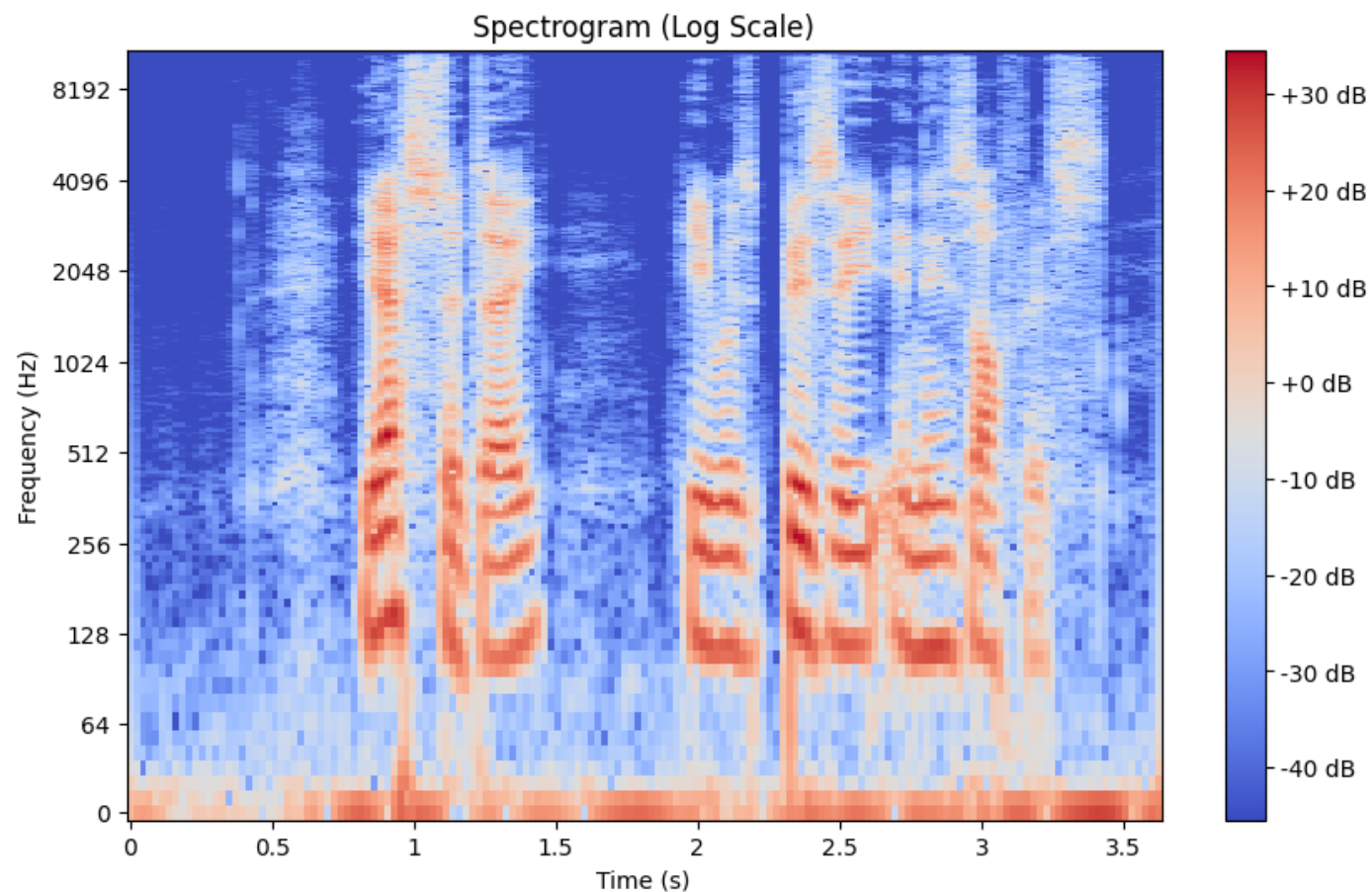
```
array([[ 8.176865, 12.681932, 13.495272, ..., 18.527195, 21.13016 , 19.365791 ],
       [ 6.5787787, 7.967007 , 7.1565847, ..., 13.040471, 16.229465 , 17.005955 ],
       [ 1.926607 , -3.0728698, -7.9710507, ..., -14.2416525, 1.917774 , 9.74291 ],
       ...,
       [-45.454266 , -45.454266 , -45.454266 , ..., -45.454266 , -45.454266 , -45.454266 ],
       [-45.454266 , -45.454266 , -45.454266 , ..., -45.454266 , -45.454266 , -45.454266 ],
       [-45.454266 , -45.454266 , -45.454266 , ..., -45.454266 , -45.454266 , -45.454266 ]], dtype=float32)
```

プロット(spectrogram)

```
plt.figure(figsize=(10, 6))
librosa.display.specshow(log_spectrogram, sr=sr, hop_length=512, x_axis="time", y_axis="log")
plt.colorbar(format="%+2.0f dB")
plt.title("Spectrogram (Log Scale)")
plt.xlabel("Time (s)")
plt.ylabel("Frequency (Hz)")
plt.show()
```

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築



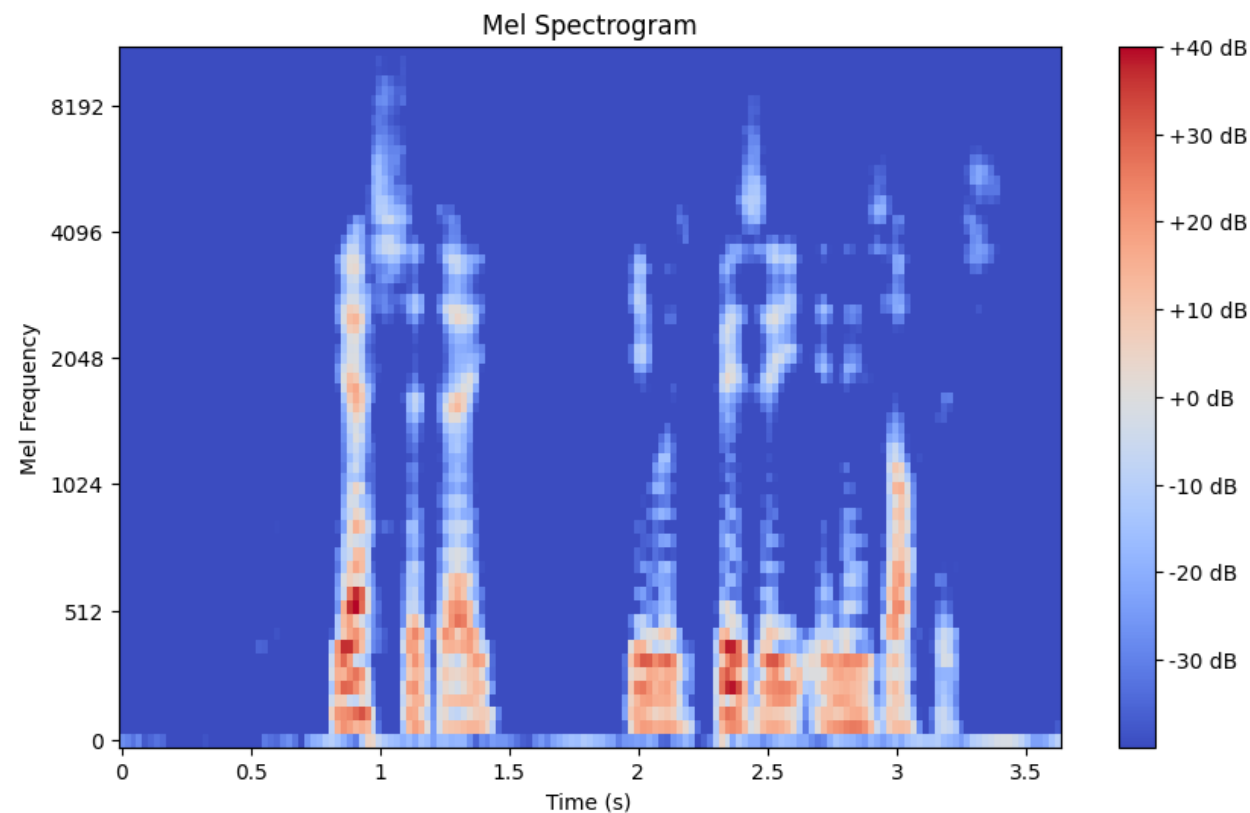
3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

プロット(melspectrogram)

```
mel_spectrogram = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=2048, hop_length=512, n_mels=64)
log_mel_spectrogram = librosa.amplitude_to_db(mel_spectrogram) # dBスケールに変換

plt.figure(figsize=(10, 6))
librosa.display.specshow(log_mel_spectrogram, sr=sr, hop_length=512, x_axis="time", y_axis="mel")
plt.colorbar(format="%+2.0f dB")
plt.title("Mel Spectrogram")
plt.xlabel("Time (s)")
plt.ylabel("Mel Frequency")
plt.show()
```



3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

(おまけ) Wav2vec2.0で音声の特徴量を得る

インポート

```
from transformers import Wav2Vec2Processor, Wav2Vec2Model
import torchaudio
```

Wav2vec2.0をファインチューニングするのは少し難しいので、今回はファインチューニング済みモデルを使用して特徴量を抽出して別のNNで分類するための学習を行ってみます

wav2vec2-base-960hは
16kHz でサンプリングされた音声オーディオで 960 時間の Librispeech で事前トレーニングおよびファインチューニングされている
モデルへの音声入力 は 16KHz にリサンプリングする

モデルの読み込みと音声の前処理

```
processor = Wav2Vec2Processor.from_pretrained("facebook/wav2vec2-base-960h")
model = Wav2Vec2Model.from_pretrained("facebook/wav2vec2-base-960h")

audio, sr = torchaudio.load(audio_path)
x = torchaudio.transforms.Resample(sr, target_sampling_rate)(audio[0]).numpy()
x = processor(x, sampling_rate=16000, return_tensors="pt") #wavファイルをwav2vecに入れる前の処理 pt:pytorchのテンソル
x = model(**x).last_hidden_state #wav2vecにwavファイルを入力
x
```

.last_hidden_stateではモデルの推論時の重み(パラメータ)の最終層を取得している

この時、出力層の値の長さは音声の長さによって変化するため、len(x[0])は可変となる

len(x[0][0]), len(x[0][1]), ...は常にすべて768となる
(音声のある時間幅で切り出した時、その切り出した音声に対する特徴量が768次元で表現されていると考えられる)

Wav2vec2.0

音声データから特徴量を抽出するための自己教師あり学習モデル(ラベルなしデータで事前学習を行った後ラベルつきデータでファインチューニングを行う)

音声データを特徴エンコーダ(Conv)で特徴量に変換し、量子化、一部をマスクしてコンテキストネットワーク(Transformer)に入力、マスクしたタイムステップの出力とマスクした特徴量で対照損失を計算することで学習する

文字起こしなどの音声認識で使用される

Wav2vec2.0は音響情報をあまり含んでいないとされているため、音声分類モデルにあまり適していない

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

speaker-info.txt

ID	AGE	GENDER	ACCENTS	REGION	COMMENTS
p225	23	F	English	Southern England	
p226	22	M	English	Surrey	
p227	38	M	English	Cumbria	
p228	22	F	English	Southern England	
p229	23	F	English	Southern England	
p230	22	F	English	Stockton-on-tees	
p231	23	F	English	Southern England	
p232	23	M	English	Southern England	
p233	23	F	English	Staffordshire	
p234	22	F	Scottish	West Dumfries	
p236	23	F	English	Manchester	
p237	22	M	Scottish	Fife	
p238	22	F	NorthernIrish	Belfast	
p239	22	F	English	SW England	
p240	21	F	English	Southern England	
p241	21	M	Scottish	Perth	
p243	22	M	English	London	
p244	22	F	English	Manchester	
p245	25	M	Irish	Dublin	
p246	22	M	Scottish	Selkirk	
p247	22	M	Scottish	Argyll	
p248	23	F	Indian		
p249	22	F	Scottish	Aberdeen	
p250	22	F	English	SE England	
p251	26	M	Indian		
p252	22	M	Scottish	Edinburgh	
p253	22	F	Welsh	Cardiff	
p254	21	M	English	Surrey	
p255	19	M	Scottish	Galloway	
p256	24	M	English	Birmingham	
p257	24	F	English	Southern England	
p258	22	M	English	Southern England	
p259	23	M	English	Nottingham	
p260	21	M	Scottish	Orkney	
p261	26	F	NorthernIrish	Belfast	
p262	23	F	Scottish	Edinburgh	
p263	22	M	Scottish	Aberdeen	
p264	23	F	Scottish	West Lothian	

CSTR **VCTK-Corpus-0.92**を使用

Edinburgh大学のCSTR（Centre for Speech Technology Research）が公開した、音声データとテキストトランスクリプトからなる大規模な多話者音声データセット

109人の話者（男女、異なるアクセント）による英語の音声データ（各話者が数百の文を話した音声）

話者は様々な英語アクセントを持ち、世界中の地域（イギリス、アメリカ、インドなど）を代表している

ファイル形式：WAV形式（48 kHz / 16 bit PCM）
 ライセンス：商用および非商用の研究に自由に使用できるが、データセットを使用する際には適切なクレジットが必要

例えばこんな分類モデル

- 性別
（男か女か 2 クラス分類）
- アクセント
（EnglishかScottishかAmericanか 3 クラス分類）
- これらを組み合わせ
性別・アクセントの組み合わせ（6 クラス分類）

例えばこんな分類モデル

3. 実際にやってみよう

1. 簡単な分類モデルを構築
2. 簡単な回帰モデルを構築
3. 音声から特徴量を抽出
4. 話者分類モデルを構築

- スペクトログラムを2次元配列として入力するFNNを用いた分類モデル
- スペクトログラムを画像として入力するCNNを用いた分類モデル
- Wav2vec2.0で抽出した特徴量を入力するFNNを用いた分類モデル

そのほか工夫できるポイント

- 学習時に何を正解ラベルとするか
- モデルの構造や学習時のパラメータ